

THE INFINITE LOOP

Relatório do Jogo de Sobrevivência em RPG de Texto

1 INTRODUÇÃO

Fizemos um jogo de sobrevivência em Python chamado The Infinite Loop. É um RPG de texto, ou seja, o jogador lê a história e digita comandos no terminal para jogar. O jogo tem 51 fases, e o objetivo é tomar as decisões certas até chegar ao final e vencer. A história foi criada por nós mesmos. Escolhemos o gênero de sobrevivência porque era o desafio mais difícil entre as opções que o professor deu.

2 OBJETIVOS

Nosso objetivo foi criar um jogo completo e divertido, que fizesse o jogador imaginar a história, os lugares, os monstros e os personagens. Também queríamos praticar Python, treinar a lógica de programação e aprender a escrever uma história junto com um jogo.

3 TECNOLOGIAS UTILIZADAS

O jogo foi feito só em Python, usando bibliotecas que já vêm na linguagem: os, io, time, random, platform e contextlib. Não usamos nenhuma biblioteca externa. Pensamos em separar o código em vários arquivos, mas decidimos deixar tudo em um arquivo só, porque ainda não sabíamos organizar um projeto assim.

4 DESENVOLVIMENTO DO JOGO

Nos inspiramos em jogos clássicos de RPG de texto, como Zork. O jogador controla tudo por comandos digitados, tanto no menu principal quanto durante a partida. No começo, ele escolhe um nome e uma raça (Humano, Elfo, Anão, Goblin ou Draconato), cada uma com atributos e um bônus diferente.

O combate é por turnos: o jogador pode atacar, fugir ou usar um item. O dano depende do ataque de quem bate, da defesa de quem apanha, e tem uma parte de sorte e chance de crítico, além de uma chance de errar o golpe, que depende da velocidade dos dois. Durante a luta pode acontecer veneno, fogo ou doença, que causam dano com o tempo.

O jogador pode usar uma arma e seis peças de armadura ao mesmo tempo. Também existe um sistema de fome, que vai diminuindo e, se chegar a zero, o personagem começa a perder vida. Os itens são conseguidos matando monstros, comprando em lojas, achando em baús ou fabricando. O jogo tem 51 fases divididas em 4 atos: Floresta, Minas, Ruínas Arcanas e Castelo, terminando na luta final contra o Lorde do Loop.

5 PROGRAMAÇÃO

O código tem cerca de 3.260 linhas em um único arquivo, com quase 65 funções, cada uma cuidando de uma parte do jogo (menus, personagem, inventário, combate, fases). Usamos entrada de texto no terminal, com if/elif/else e laços de repetição para validar o que o jogador digita.

O inventário é guardado em um dicionário, com um menu de 8 opções (usar, fabricar, descartar, ordenar, ver detalhes, equipar, buscar e sair). Os itens têm peso, valor e descrição, e o sistema

de fabricação (crafting) confere se o jogador tem os materiais antes de deixar fabricar. Para deixar o terminal mais bonito, usamos cores ANSI e um efeito de escrita lenta, como se o texto estivesse sendo digitado na hora. Aplicamos os conceitos que aprendemos em aula: variáveis, condicionais, laços, funções, listas e dicionários.

6 DIFICULDADES E SOLUÇÕES

O tempo foi curto para fazer tudo que planejamos. Também usamos coisas que ainda não tínhamos visto em aula, como dicionários, cores ANSI, a biblioteca time e números aleatórios, então tivemos que aprender sozinhos. O sistema de combate foi o mais difícil, por ter muitas partes conectadas, e o código foi ficando grande e bagunçado, mas não deu tempo de reorganizar em vários arquivos.

Para resolver, pesquisamos no Google, no YouTube e usamos IAs (Grok, Gemini e Claude) para entender as cores ANSI e a biblioteca time, e também para revisar a ortografia dos textos das fases. Toda a história e a narrativa foram escritas por nós; a IA só ajudou na revisão e em dúvidas de código.

7 RESULTADOS

Ficamos satisfeitos com o resultado final, que continua fiel à ideia original, mesmo com algumas mudanças no caminho. O jogo tem sistema de menu, criação de personagem, status completo, inventário, equipamento, crafting, combate por turnos, fome, quatro vendedores diferentes, progressão por XP e nível, e recursos visuais como cores e efeito de escrita lenta.

Como pontos de melhoria, o combate poderia ter mais opções de ação, o visual do terminal poderia ser mais elaborado, e poderiam existir mais itens, receitas e inimigos. Alguns bugs ainda podem ser corrigidos no futuro.

8 CONCLUSÃO

Conseguimos aplicar tudo que aprendemos em aula e ainda desenvolvemos habilidades novas sozinhos. O resultado superou o que esperávamos, e ficamos orgulhosos, porque nunca imaginamos conseguir fazer um jogo tão complexo. Foi a experiência mais divertida que já tivemos com Python. No futuro, queremos melhorar o código com o que formos aprendendo no curso técnico.

REFERÊNCIAS

APOENA STACK. Tutoriais de programação. YouTube. Disponível em: <https://youtu.be/yfAixrxCVfo>. Acesso em: 15 ago. 2026.

CURSO EM VÍDEO. Python 3 – Mundo 1. YouTube. Disponível em: <https://youtu.be/0hBIhkcA8O8>. Acesso em: 15 ago. 2026.

CURSO EM VÍDEO. Python 3 – Mundo 2. YouTube. Disponível em: <https://youtu.be/ZWj8o692qGY>. Acesso em: 15 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Python.org. Disponível em: <https://www.python.org/>. Acesso em: 15 ago. 2026.